

Atividade 3 – Sistema Cliente/Servidor em Camadas para Processamento de Áudio

Objetivo

Implementar um sistema cliente/servidor em três camadas capaz de enviar, processar e armazenar **arquivos de áudio de forma organizada**.

Arquitetura

1. Cliente (GUI com PySide6)

- Seleciona um arquivo de áudio.
- Envia o áudio via HTTP para o servidor, indicando o processamento desejado.
- Permite reproduzir o áudio original e o processado.
- Exibe histórico de arquivos já enviados e processados.
- Exibe informações básicas do áudio, como duração, formato e tamanho.

2. Servidor (FastAPI + FFmpeg)

- Recebe arquivos de áudio enviados pelo cliente.
- Aplica operações de processamento, por exemplo:
 - Normalização de volume.
 - Conversão para mono.
 - Alteração da velocidade de reprodução.
 - Redução da taxa de bits.
 - Conversão de formato.
- Armazena os arquivos em pastas organizadas por **data** e **UUID**.
- Registra metadados no banco de dados.

- Disponibiliza uma interface web simples para listar os áudios armazenados, permitindo sua reprodução diretamente pelo navegador.

3. Banco de Dados (PostgreSQL)

Tabela **audios** com os campos (sugestões):

- id (UUID, chave primária)
- original_name
- original_ext
- mime_type
- size_bytes
- duration_sec
- sample_rate
- channels
- bitrate
- processing_type
- created_at
- path_original
- path_processed

Regras

- Cada arquivo de áudio possui um **UUID único**.
- O nome do arquivo armazenado é sempre **audio.{ext}**, organizado em subpastas.
- Metadados complementares ficam armazenados em **meta.json**.
- O arquivo **meta.json** pode conter informações como checksum, parâmetros utilizados no processamento e tamanho do arquivo.
- Uma imagem da **forma de onda do áudio** (**waveform.png**) é gerada automaticamente para visualização rápida.
- O diretório **trash/** armazena temporariamente arquivos marcados para exclusão.

Fluxo de Funcionamento

1. Cliente seleciona um arquivo de áudio e escolhe o processamento desejado.
2. Cliente envia o arquivo para o servidor através de uma requisição HTTP.
3. Servidor recebe o áudio e gera um UUID para identificá-lo.
4. Servidor processa o arquivo utilizando FFmpeg.
5. Servidor salva o áudio original e o áudio processado em disco.
6. Metadados são registrados no banco PostgreSQL.
7. Servidor gera a representação gráfica da forma de onda do áudio.
8. Histórico pode ser consultado pelo cliente através da API.
9. Cliente pode reproduzir tanto o arquivo original quanto o processado.

Demonstração

- Selecionar um arquivo de áudio no computador cliente.
- Escolher um processamento, por exemplo **normalização de volume**.
- Enviar o arquivo para o servidor.
- Servidor realiza o processamento.
- Reproduzir o áudio original.
- Reproduzir o áudio processado.
- Consultar o histórico de arquivos enviados.
- Consultar no banco os metadados armazenados.
- Mostrar a organização dos arquivos no servidor conforme a estrutura especificada.
- Mostrar a interface web do servidor com os arquivos de áudio disponíveis.

Tecnologias

- **Python 3**
- **FastAPI** (API HTTP)
- **PySide6** (cliente gráfico)
- **FFmpeg** (processamento de áudio)
- **PostgreSQL** (armazenamento de metadados)
- **SQLAlchemy** (comunicação entre servidor e banco de dados)
- **Dois computadores distintos** (cliente e servidor)

Entrega

- Repositório no GitHub com:
- Código-fonte do cliente.
- Código-fonte do servidor.
- Scripts ou configuração do banco de dados.
- **README .md** com:
 - Descrição do projeto.
 - Arquitetura utilizada.
 - Instruções de instalação.
 - Instruções de execução do cliente.
 - Instruções de execução do servidor.
 - Configuração do banco de dados.
 - Exemplos de processamento disponíveis.
 - Prints da interface.
 - Prints da organização dos arquivos.
 - Vídeo demonstrativo opcional.